

Assignment-2 (25 marks)

Assignment-2: Agenda

- Overview
 - Description and submission
- Part-1: HDL
 - Design, validation, deliverables, and grading
- Part-2: HLS & RISC-V
 - Design, validation, deliverables, and grading

Assignment-2: Overview

Overview: Description

- Objective: Learn about HDL generation/writing and RISC-V ISA
- Tools: Vivado, Vitis HLS, Godbolt compiler explorer

Description (details in subsequent slides)	Max. Marks
<u>Part-1</u> : Implement matrix multiplication directly in HDL (Vivado)	10
<u>Part-2</u> : Implement matrix multiplication & convolution in C++ Convert both to HDL (Vitis) and RISC-V assembly (Godbolt)	10
<u>Part-3</u> : In-class quiz (will be introduced separately later)	05

Overview: Submission

- **Deadline: 21 Oct 2026 (23:59)**
- Only first two parts of the assignment need submission
- Submission must be a single .zip file
- How to organize this zip file?
 - Make separate folders for deliverables of Part-1 & Part-2
 - Combine these parts into a single zip file
 - Name this combined file as FirstName_LastName_EntryNo_A2.zip
 - **Penalty of 2 marks for not following the above naming convention**

Assignment-2: Part-1 (HDL)

HDL: Agenda

- Design
- Validation
- Deliverables
- Grading

HDL: Design

- Implement $A*B = C$ using any HDL
 - A ($m \times n$), B ($n \times k$), & C ($m \times k$) are matrices
- You are allowed to use FPGA IPs
- You can use only **signed** data types for matrix elements
 - Any signed data format (int, float etc.) & any bit-width is allowed
- **Do not hardcode** dimensions of A & B matrices (m, n, k above)
 - Hint: use parameter feature in HDLs to initialize them
- Recommended: align this design with your project for reuse

HDL: Validation

- You have to validate your design by waveform simulation
- Set A & B dimensions to 8 x 8 elements for simulation
 - You can initialize matrix elements to any non-zero distinct values
- Take screenshots of the waveform showing
 - Any 3 non-zero output values
 - Indices of these 3 values in output matrix - flattened/1D index is allowed (Hint: Vivado radix change feature shows waveform values in decimal)
- Justify that the screenshot output matches manual calculation
 - Values and indices of the 3 output values selected earlier

HDL: Deliverables

- HDL files
 - Design code & testbench
- One PDF file that contains
 - Text (1-2 lines): HDL name, data format & bit-size, parameter construct
 - Screenshot/s showing 3 non-zero output values & their indices
 - Manual calculation of the full output matrix in decimal (just values)
 - Justification (2-3 lines) that 3 output values & indices match manual calc.
 - One interesting observation/challenge about design/validation process + its analysis (no limit on lines, but brief preferred)

HDL: Grading

- HDL files: 5 marks
- PDF file: 5 marks - breakdown below
 - Initial text description (0.5 mark)
 - Screenshot/s outputs & indices (1 mark)
 - Manual output calculation (0.5 mark)
 - Justification of output matching manual calculation (1.5 marks)
 - One interesting observation/challenge + its analysis (1.5 marks)

Assignment-2: Part-2 (HLS & RISC-V)

HLS & RISC-V: Agenda

- Design
- Validation
- Deliverables
- Grading

HLS & RISC-V: Design

- Implement matrix multiplication $A*B = C$ in C++
 - A & B dimensions as 8x8, use any signed datatype & any data values
- Implement convolution of 8x8 image with 2x2 kernel in C++
 - Both are single channel, use any signed datatype & any data values
- Convert C++ codes to any HDL using Vitis HLS
- Convert C++ codes into RISC-V assembly using Godbolt compiler
 - Compiler URL: <https://godbolt.org/>
 - Target ISA: rv32imf_zve32x_zicsr_zifencei_zbb
 - Figure out settings like “compiler” & “compiler options” for target ISA

HLS & RISC-V: Validation

- Perform below validations separately for matmul and convolution
- HLS conversion validation
 - Perform C Simulation to validate functionality of C++
Take screenshot of HLS console/report showing C Simulation “Pass”
 - Perform C-RTL co-simulation to show HDL functions like validated C++
Take screenshot of HLS console/report showing C-RTL co-simulation “Pass”
- RISC-V conversion validation
 - Figure out a way to show that assembly output matches C++
(You can use tools other than Godbolt compiler for this, if needed)
 - Take a screenshot showing assembly outputs match C++ outputs

HLS & RISC-V: Deliverables

- Files (one set for matmul and one set for convolution)
 - C++ source code & C++ testbench
 - HLS files: generated top module HDL code (HDL testbench not needed)
 - RISC-V files: generated assembly code (copy to text file if needed)
- Single PDF file
 - Screenshots of C simulation & C-RTL co-simulation as “PASS”
 - One for matmul and one for convolution
 - **One** interesting observation about HLS conversion and its analysis
 - Not mandatory to separate between matmul and convolution
 - Screenshot showing RISC-V assembly output matches C++ outputs
 - One for matmul and one for convolution
 - **One** interesting observation about RISC-V conversion and its analysis
 - Not mandatory to separate between matmul and convolution

HLS & RISC-V: Grading

- Files: 3 marks
- Single PDF file: 7 marks – breakdown below
 - Screenshots of C sim & C-RTL co-sim “PASS” (2 marks)
 - Interesting observation about HLS & its analysis (1.5 marks)
 - Screenshot of RISC-V vs C++ output match (2 marks)
 - Interesting observation about RISC-V and its analysis (1.5 marks)